

RAG Mongo Demo v9

Project Overview

RAG Mongo Demo v9 is a full-stack retrieval-augmented generation workspace for healthcare test cases and user stories. It supports Excel-to-JSON conversion, embedding creation, MongoDB Atlas Vector Search, MongoDB Atlas Search BM25, hybrid retrieval, Groq-based reranking and summarization, and prompt/schema management for downstream generation workflows.

The application is split into a React front end and an Express/MongoDB back end, with shared utility scripts under `src/scripts` for ingestion, preprocessing, search experiments, and AI integration.

Tech Stack

| Layer | Technologies |

| --- | --- |

| Front end | React 19, React Router, Material UI, MUI X Data Grid, Notistack, Axios |

| Back end | Node.js, Express, MongoDB Node driver, CORS, Multer, Dotenv, Child Process orchestration |

| Data and search | MongoDB Atlas, Atlas Vector Search, Atlas Search BM25, Excel/XLSX processing |

| AI and LLM integration | Mistral Embeddings API, Groq Chat Completions API, Testleaf/OpenAI-compatible embedding API, Hugging Face Inference API |

Prerequisites

Install the following before running the project:

- Node.js 18 or later

- npm 9 or later
- MongoDB Atlas cluster with the target database and collections
- Atlas Search indexes for BM25 and Vector Search
- API keys and environment variables for Mistral, Groq, and optional Jira/Testleaf/Hugging Face integrations

Required Environment Variables

The app reads configuration from ``.env`` in the project root.

- ``MONGODB_URI``
- ``DB_NAME``
- ``COLLECTION_NAME``
- ``VECTOR_INDEX_NAME``
- ``BM25_INDEX_NAME``
- ``USER_STORIES_COLLECTION_NAME``
- ``USER_STORIES_VECTOR_INDEX_NAME``
- ``USER_STORIES_BM25_INDEX_NAME``
- ``MISTRAL_API_KEY``
- ``MISTRAL_EMBEDDING_MODEL``
- ``GROQ_API_KEY``
- ``GROQ_RERANK_MODEL``
- ``GROQ_SUMMARIZATION_MODEL``
- ``TESTLEAF_API_BASE``
- ``USER_EMAIL``
- ``AUTH_TOKEN``
- ``JIRA_BASE_URL``
- ``JIRA_EMAIL``

- `JIRA\_API\_TOKEN`
- `JIRA\_PROJECT\_KEY`
- `HUGGINGFACE\_API\_KEY` or `HF\_TOKEN`

Run Commands

From the project root:

```
``bash
npm install
npm run dev
``
```

Useful commands:

```
``bash
npm run server # Start Express API on port 3001
npm run client # Start React app in the client folder
npm run build  # Build the React front end for production
``
```

`npm install` at the root also triggers the client install through the root `postinstall` script.

Key Features

- Excel upload and conversion to JSON for test cases and user stories
- Background embedding creation with job tracking and progress polling
- Query preprocessing with normalization, abbreviation expansion, and synonym expansion

- Vector search over test cases and user stories using Mistral embeddings
- BM25 keyword search with field weighting and Atlas Search highlights
- Hybrid search that fuses BM25 and vector scores
- Groq-powered reranking and summarization workflows
- Duplicate detection and summary generation for result sets
- Prompt template and JSON schema management for AI generation flows
- Environment variable editing from the UI

Architecture Flow

```mermaid

flowchart LR

U[User] --> R[React UI]

R --> E[Express API]

E --> C1[Excel Conversion]

E --> C2[Query Preprocessing]

E --> C3[Search APIs]

E --> C4[Prompt/Test Utilities]

E --> C5[Settings / Env]

C1 --> F[uploads/ and src/data/]

C2 --> P[src/scripts/query-preprocessing]

C3 --> M[MongoDB Atlas]

C3 --> Mistral[Mistral Embeddings]

C3 --> Groq[Groq Rerank / Summarize]

C3 --> HF[Optional Hugging Face Rerank]

M --> BS[BM25 Index]

M --> VS[Vector Index]

M --> US[User Stories Collection]

R --> FE[Convert, Embeddings, Search, Rerank, Summarize, Prompt Schema, Settings]

...

## ## Front End Files and Usage

| File | Purpose |

| --- | --- |

| `client/src/index.js` | React bootstrap file that mounts the app into the DOM. |

| `client/src/App.js` | Main navigation shell, theme switcher, sidebar, and section routing. |

| `client/src/App.css` | Legacy app-level styling for the React shell. |

| `client/src/index.css` | Global CSS reset and base typography styles. |

| `client/src/components/data/ConvertToJson.js` | Uploads Excel files and calls `/api/upload-excel` to convert test case or user story sheets into JSON. |

| `client/src/components/data/EmbeddingsStore.js` | Lists generated JSON files, polls embedding jobs, and starts batch embedding creation. |

| `client/src/components/processing/QueryPreprocessing.js` | Runs query normalization, abbreviation expansion, and synonym expansion through `/api/search/preprocess` and `/api/search/analyze`. |

| `client/src/components/processing/SummarizationDedup.js` | Executes search, deduplication, and summarization workflows through `/api/search/deduplicate` and `/api/search/summarize`. |

| `client/src/components/processing/PromptSchemaManager.js` | Manages prompt templates, JSON schema, prompt testing, and RAG workflow demonstrations. |

| `client/src/components/search/QuerySearch.js` | Vector search UI with metadata filters and similarity scoring. |

| `client/src/components/search/BM25Search.js` | BM25 keyword search UI with Atlas Search filters and score breakdown. |

| `client/src/components/search/HybridSearch.js` | Combined BM25 + vector search UI with adjustable score weights. |

| `client/src/components/search/RerankingSearch.js` | Groq AI reranking UI with before/after comparison views. |

| `client/src/components/settings/Settings.js` | Reads and updates `.env` values through `/api/env`. |

## ## Back End Files and Usage

| File | Purpose |

| --- | --- |

| `server/index.js` | Main Express API. Hosts health checks, uploads, job tracking, environment editing, preprocessing, deduplication, summarization, search, reranking, and user-story retrieval. |

| `src/config/testcases-bm25-index.json` | Atlas Search definition for BM25 over test case documents. |

| `src/config/testcases-vector-index.json` | Atlas Vector Search definition for the test case embedding field. |

| `src/config/user-stories-bm25-index.json` | Atlas Search definition for BM25 over user story documents. |

| `src/config/user-stories-vector-index.json` | Atlas Vector Search definition for the user story embedding field. |

| `src/data/\*.json` | Generated JSON outputs created from Excel or story ingestion runs. |

| `uploads/` | Temporary upload storage for incoming Excel files. |

## ## Integration and Utility Files

### ### Data Conversion

| File | Purpose |

| --- | --- |

| `src/scripts/data-conversion/excel-to-json.js` | Converts test case Excel sheets into JSON documents for ingestion. |

| `src/scripts/data-conversion/excel-to-userstories.js` | Converts user story Excel sheets into Jira-like JSON structure. |

| `src/scripts/data-conversion/fetch-jira-stories.js` | Pulls stories from Jira via REST API and writes normalized JSON. |

### ### Embeddings

| File | Purpose |

| --- | --- |

| `src/scripts/embeddings/create-embeddings-batch-mistral.js` | Batch-creates test case embeddings using Mistral. |

| `src/scripts/embeddings/create-userstories-embeddings-batch-mistral.js` | Batch-creates user story embeddings using Mistral. |

### ### Query Preprocessing

| File | Purpose |

| --- | --- |

| `src/scripts/query-preprocessing/normalizer.js` | Normalizes text, tokenizes queries, and extracts test case IDs. |

| `src/scripts/query-preprocessing/abbreviationMapper.js` | Expands healthcare and test-case abbreviations. |

| `src/scripts/query-preprocessing/synonymExpander.js` | Builds synonym and phrase variations for search expansion. |

| `src/scripts/query-preprocessing/dictionaries.js` | Source dictionaries for abbreviations, synonyms, and phrase mappings. |

| `src/scripts/query-preprocessing/queryPreprocessor.js` | Orchestrates the preprocessing pipeline. |

| `src/scripts/query-preprocessing/test-preprocessing.js` | CLI test harness for preprocessing validation. |

### ### Search and Ranking

| File | Purpose |

| --- | --- |

| `src/scripts/search/search-vector-db.js` | Standalone vector search against the test case collection using Testleaf/OpenAI-compatible embeddings. |

| `src/scripts/search/search-jira-stories.js` | Standalone vector search for Jira-derived user stories. |

| `src/scripts/search/search-combined-stores.js` | Searches both test cases and user stories, then merges the results. |

| `src/scripts/search/score-fusion-search.js` | Demonstrates score fusion between BM25 and vector results. |

| `src/scripts/search/rerank-search.js` | Standalone reranking workflow using Hugging Face CrossEncoder. |

| `src/scripts/search/bm25-search.js` | Standalone BM25 search with field weighting. |

| `src/scripts/search/compare-field-weights.js` | Compares alternative BM25 field-weight configurations. |

### ### Support Utilities

| File | Purpose |

| --- | --- |

| `src/scripts/utilities/mistralEmbedding.js` | Shared Mistral embedding helper with retry logic and batch support. |

| `src/scripts/utilities/groqClient.js` | Shared Groq client for reranking, summarization, and answer generation. |

| `src/scripts/utilities/delete-all-documents.js` | Utility to clear the configured MongoDB collection. |



### ### Story and Release Assets

| File | Purpose |

| --- | --- |

| `releases/1.21.2/stories/\*.txt` | Sample release story inputs used for traceability and testing. |

### ## APIs Created and How They Work

| API | How it works |

| --- | --- |

| `GET /api/health` | Lightweight health check that returns the server status. |

| `GET /api/jobs/active` | Returns in-progress embedding jobs from the in-memory job store. |

| `GET /api/jobs/:jobId` | Returns the current status, progress, and results for a specific job. |

| `GET /api/metadata/distinct` | Connects to MongoDB, inspects the collection, and returns distinct metadata values for filters. |

| `GET /api/files` | Lists JSON files in `src/data` so the UI can select generated artifacts. |

| `POST /api/upload-excel` | Accepts an Excel upload, converts it to JSON, and saves the output in `src/data`. |

| `POST /api/create-embeddings` | Starts a background job that creates embeddings file-by-file. |

| `POST /api/create-embeddings-batch` | Starts the faster batch embedding flow using the appropriate batch script. |

| `GET /api/env` | Reads the local `.env` file and returns editable environment values. |

| `POST /api/env` | Writes updated environment values back to `.env`. |

| `POST /api/search/preprocess` | Runs the query preprocessing pipeline and returns transformed query variants. |

| `POST /api/search/analyze` | Returns preprocessing diagnostics without performing the transformation. |

| `POST /api/search/deduplicate` | Deduplicates search results using title similarity. |

| `POST /api/search/summarize` | Sends result text to Groq and returns a summary plus token/cost metadata. |

| `POST /api/test-prompt` | Sends an arbitrary prompt to Groq and returns parsed output, usage, and cost. |

| `POST /api/search` | Executes vector search using Mistral embeddings and MongoDB Atlas Vector Search. |

| `POST /api/search/bm25` | Executes Atlas Search BM25 keyword search with filtering and field selection. |

| `POST /api/search/hybrid` | Runs BM25 and vector retrieval, normalizes scores, and fuses them into a hybrid ranking. |

| `POST /api/search/rerank` | Uses BM25 + vector candidates, then reranks them with Groq AI. |

| `GET /api/testcases/latest-id` | Reads all test case IDs and returns the next available `TC\_xxxx` value. |

| `POST /api/search/user-stories` | Searches the user story collection with vector and optional BM25 retrieval, then adds impact and risk annotations. |

## ## External APIs and Integrations

| API / Service | Used for |

| --- | --- |

| MongoDB Atlas Search | BM25 retrieval, highlights, and filterable keyword search. |

| MongoDB Atlas Vector Search | Semantic retrieval over embedded test cases and user stories. |

| Mistral Embeddings API | Generating 1024-dimensional embeddings with `mistral-embed`. |

| Groq Chat Completions API | Document reranking, summarization, and prompt testing. |

| Testleaf/OpenAI-compatible embedding API | Standalone scripts that generate `text-embedding-3-small` embeddings. |

| Jira REST API | Pulling user stories into normalized JSON. |  
| Hugging Face Inference API | Optional CrossEncoder reranking for search experiments. |

## ## LLM Models Used by Section

| Section | Model / Provider | Role |  
| --- | --- | --- |  
| Embeddings creation | `mistral-embed` via Mistral | Primary embedding model for test cases and user stories. |  
| Vector search | `mistral-embed` via Mistral | Embeds search queries before MongoDB Vector Search. |  
| Hybrid search | `mistral-embed` via Mistral | Query embedding for the vector half of the fused retrieval flow. |  
| Groq reranking | `llama-3.2-3b-preview` via Groq | Fast, low-cost ranking of candidate documents. |  
| Summarization | `llama-3.3-70b-versatile` via Groq | Higher-quality summary generation. |  
| Prompt testing | `llama-3.2-3b-preview` via Groq | Response generation for prompt experiments. |  
| User-story impact analysis | `mistral-embed` via Mistral | Retrieves similar user stories for impact and risk annotation. |  
| Standalone search scripts | `text-embedding-3-small` via Testleaf/OpenAI-compatible API | Query embedding in CLI search experiments. |  
| Optional reranking experiment | `cross-encoder/ms-marco-MiniLM-L-6-v2` via Hugging Face | Cross-encoder reranking in standalone scripts. |

## ## Notes on Generated Artifacts

- `client/build/` contains the production React build output.
- `src/data/converted-\*.json` and `src/data/stories-\*.json` are generated conversion artifacts.
- `uploads/` is temporary and may be cleared between runs.

## **## Suggested Workflow**

1. Configure ``.env`` and create the required MongoDB Atlas indexes.
2. Run ``npm install`` and then ``npm run dev``.
3. Convert Excel input files through the UI.
4. Generate embeddings for the converted documents.
5. Run vector, BM25, hybrid, or reranked search from the UI.
6. Use the preprocessing, deduplication, summarization, and prompt/schema tools as needed.